

Adventist University of Central Africa

Master's in Big Data Analytics

Chapter 3 — Linear Regression Lab

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics
Course Title: Data Mining
Course Code: MSDA 9124
Lecturer: Dr. Pacifique Nizeyimana
Student Name: Wayne Rubangisa
Student ID: 101220

September 2026

1 Introduction

This lab walks through fitting and interpreting linear regression models in R, following the workflow used in Chapter 3 of *An Introduction to Statistical Learning*. We'll cover:

- Simple linear regression
- Confidence and prediction intervals
- Diagnostic plots for checking model assumptions
- Multiple linear regression
- Interaction terms
- Non-linear transformations of predictors
- Qualitative (categorical) predictors
- Writing a small reusable function

We'll use two classic datasets that ship with the ISLR2 package: `Boston` (housing values in the suburbs of Boston) and `Carseats` (simulated child car seat sales data).

1.1 Required packages

```
# install.packages(c("MASS", "ISLR2", "car")) # uncomment if needed
library(MASS)      # for some classic datasets and stats utilities
library(ISLR2)     # Boston and Carseats datasets live here (2nd edition)
library(car)       # for vif() -- variance inflation factors
```

Note: In the 2nd edition of ISLR, the `Boston` dataset moved from the `MASS` package into `ISLR2`. Loading both packages avoids ambiguity issues; if you have both loaded, `ISLR2::Boston` is the one used below.

2 Simple Linear Regression

We start by exploring the relationship between `medv` (median home value, in \$1000s) and `lstat` (percentage of the population with “lower status”, e.g. lower income/education) in the `Boston` data set.

```
Boston <- ISLR2::Boston
str(Boston)
```

```
## 'data.frame':    506 obs. of  13 variables:
## $ crim      : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn        : num  18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus     : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
## $ chas      : int   0 0 0 0 0 0 0 0 0 0 ...
## $ nox       : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
## $ rm        : num  6.58 6.42 7.18 7 7.15 ...
## $ age       : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
## $ dis       : num  4.09 4.97 4.97 6.06 6.06 ...
## $ rad       : int   1 2 2 3 3 3 5 5 5 5 ...
## $ tax       : num  296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio   : num  15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
## $ lstat     : num  4.98 9.14 4.03 2.94 5.33 ...
```

```
## $ medv : num 24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

```
names(Boston)
```

```
## [1] "crim" "zn" "indus" "chas" "nox" "rm" "age"
## [8] "dis" "rad" "tax" "ptratio" "lstat" "medv"
```

2.1 Fitting the model

Before fitting anything, it's worth being explicit about what `lm()` is actually estimating. A simple linear regression assumes the true relationship has the form

$$\text{medv} = \beta_0 + \beta_1 \cdot \text{lstat} + \epsilon$$

where β_0 and β_1 are unknown population parameters and ϵ is irreducible error. `lm()` doesn't know β_0 and β_1 — it estimates them, $\hat{\beta}_0$ and $\hat{\beta}_1$, by finding the line that minimizes the residual sum of squares over the n observed pairs (x_i, y_i) :

$$\text{RSS} = \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

which has the closed-form (least-squares) solution

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

```
lm.fit <- lm(medv ~ lstat, data = Boston)
lm.fit
```

```
##
## Call:
## lm(formula = medv ~ lstat, data = Boston)
##
## Coefficients:
## (Intercept)      lstat
##      34.55      -0.95
```

```
summary(lm.fit)
```

```
##
## Call:
## lm(formula = medv ~ lstat, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.168  -3.990  -1.318   2.034  24.500
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
```

```
## (Intercept) 34.55384    0.56263    61.41    <2e-16 ***
## lstat      -0.95005    0.03873   -24.53    <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.216 on 504 degrees of freedom
## Multiple R-squared:  0.5441, Adjusted R-squared:  0.5432
## F-statistic: 601.6 on 1 and 504 DF,  p-value: < 2.2e-16
```

The output above gives us $\hat{\beta}_0$ and $\hat{\beta}_1$ (under `Estimate`), their standard errors, t-statistics, and p-values, along with two summary measures of overall fit:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}, \quad \text{RSE} = \sqrt{\frac{\text{RSS}}{n-2}}$$

R^2 is the proportion of variance in `medv` explained by `lstat` (unitless, between 0 and 1); the residual standard error (RSE) is roughly the average distance, in `medv`'s own units (\$1000s), that an observation falls from the fitted line.

We can pull out specific pieces of the fitted model:

```
names(lm.fit)
```

```
## [1] "coefficients" "residuals"      "effects"        "rank"
## [5] "fitted.values" "assign"         "qr"            "df.residual"
## [9] "xlevels"      "call"          "terms"         "model"
```

```
coef(lm.fit)
```

```
## (Intercept)      lstat
## 34.5538409   -0.9500494
```

```
confint(lm.fit)
```

```
##                2.5 %      97.5 %
## (Intercept) 33.448457 35.6592247
## lstat      -1.026148 -0.8739505
```

2.2 Confidence and prediction intervals

Given a value of `lstat`, we can produce a confidence interval for the mean response and a prediction interval for an individual observation. Both are centered on the same point prediction $\hat{y}_0 = \hat{\beta}_0 + \hat{\beta}_1 x_0$, but differ in what uncertainty they account for:

$$\text{CI: } \hat{y}_0 \pm t_{\alpha/2, n-2} \cdot \text{SE}(\hat{y}_0), \quad \text{PI: } \hat{y}_0 \pm t_{\alpha/2, n-2} \cdot \sqrt{\text{SE}(\hat{y}_0)^2 + \hat{\sigma}^2}$$

The confidence interval only accounts for uncertainty in *estimating* the regression line itself (i.e. in $\hat{\beta}_0, \hat{\beta}_1$). The prediction interval adds the irreducible error variance $\hat{\sigma}^2$ on top of that, since an individual observation also deviates randomly from the true line — which is exactly why, as the formulas above make clear, the PI is always wider than the CI at the same x_0 .

```
new_lstat <- data.frame(lstat = c(5, 10, 15))

predict(lm.fit, new_lstat, interval = "confidence")
```

```
##          fit          lwr          upr
## 1 29.80359 29.00741 30.59978
## 2 25.05335 24.47413 25.63256
## 3 20.30310 19.73159 20.87461
```

```
predict(lm.fit, new_lstat, interval = "prediction")
```

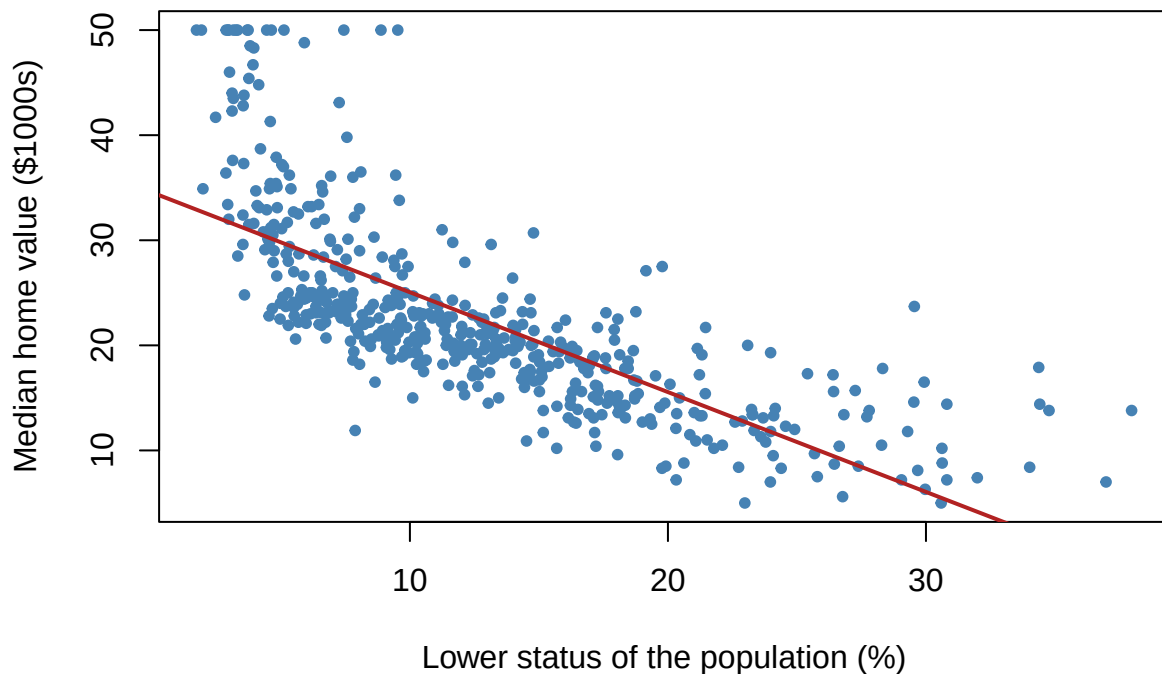
```
##          fit          lwr          upr
## 1 29.80359 17.565675 42.04151
## 2 25.05335 12.827626 37.27907
## 3 20.30310  8.077742 32.52846
```

Note that the prediction interval is wider than the confidence interval at each value of `lstat`, since it accounts for the irreducible error around the regression line, not just the uncertainty in estimating the line itself.

2.3 Visualizing the fit

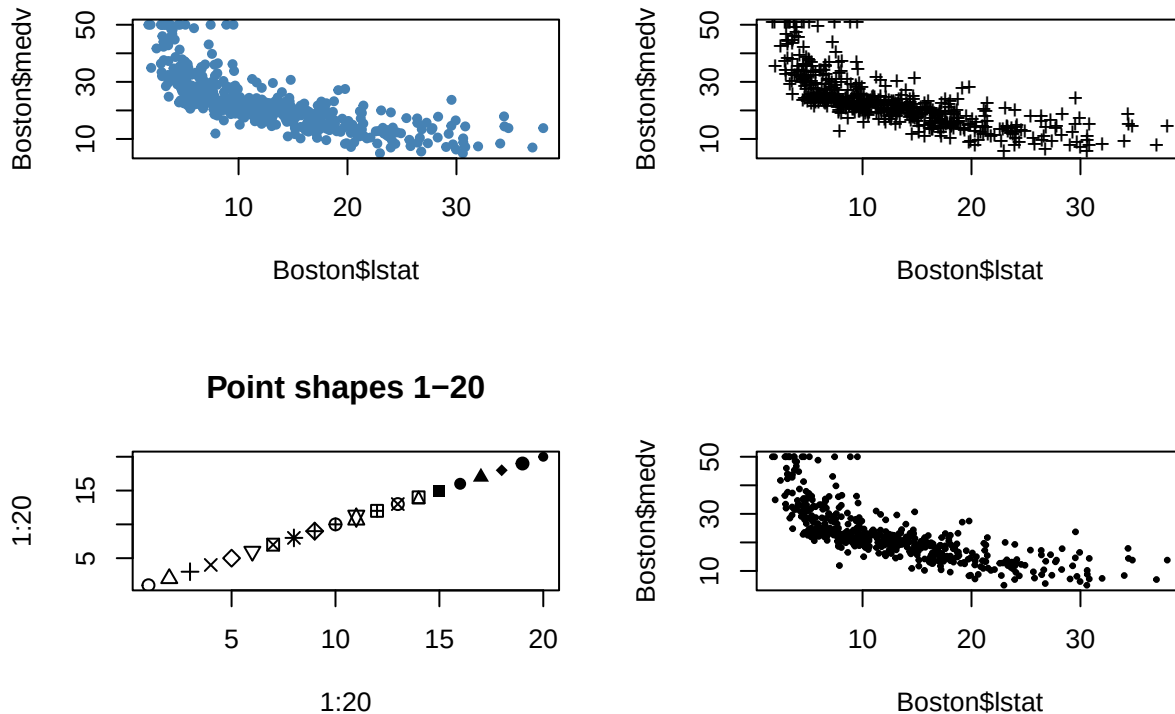
```
plot(Boston$lstat, Boston$medv,
     xlab = "Lower status of the population (%)",
     ylab = "Median home value ($1000s)",
     main = "medv vs. lstat", pch = 20, col = "steelblue")
abline(lm.fit, lwd = 2, col = "firebrick")
```

medv vs. lstat



Some extra styling options for `plot()` and `abline()`:

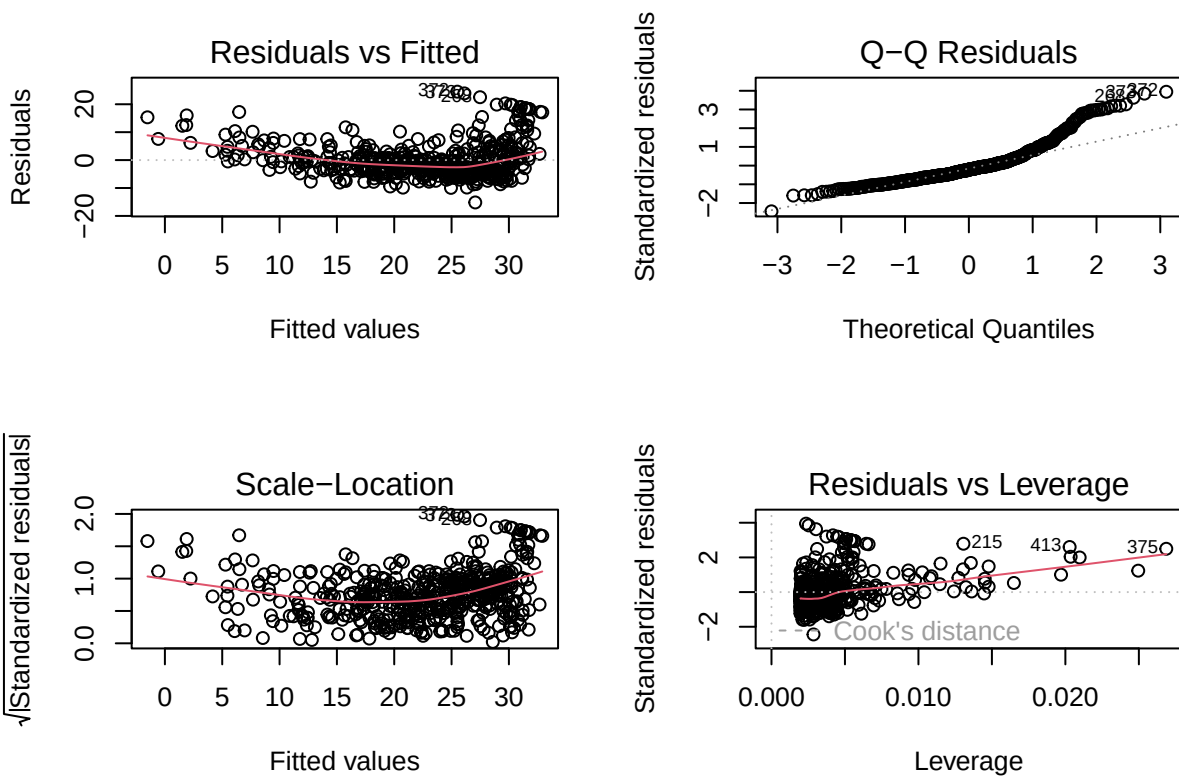
```
par(mfrow = c(2, 2))
plot(Boston$lstat, Boston$medv, col = "steelblue", pch = 20)
plot(Boston$lstat, Boston$medv, pch = "+")
plot(1:20, 1:20, pch = 1:20, main = "Point shapes 1-20")
plot(Boston$lstat, Boston$medv, pch = 20, cex = 0.6)
```



2.4 Diagnostic plots

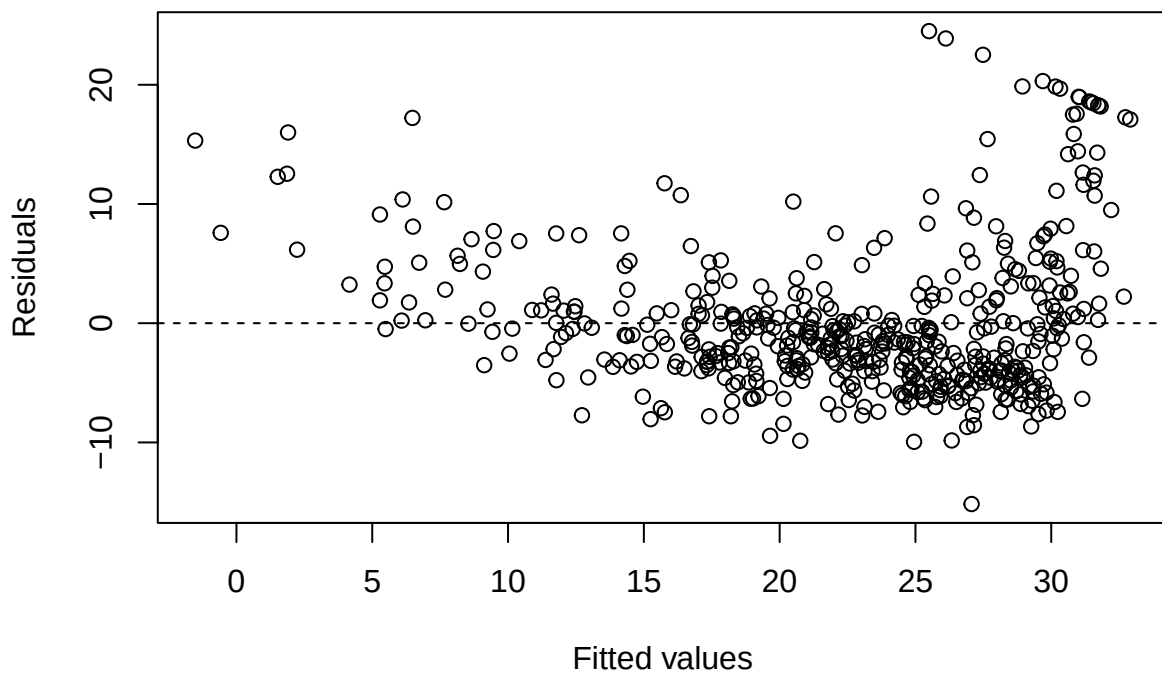
Base R's `plot()` applied directly to a fitted `lm` object produces four standard diagnostic plots: residuals vs. fitted, a Q-Q plot, scale-location, and residuals vs. leverage.

```
par(mfrow = c(2, 2))
plot(lm.fit)
```

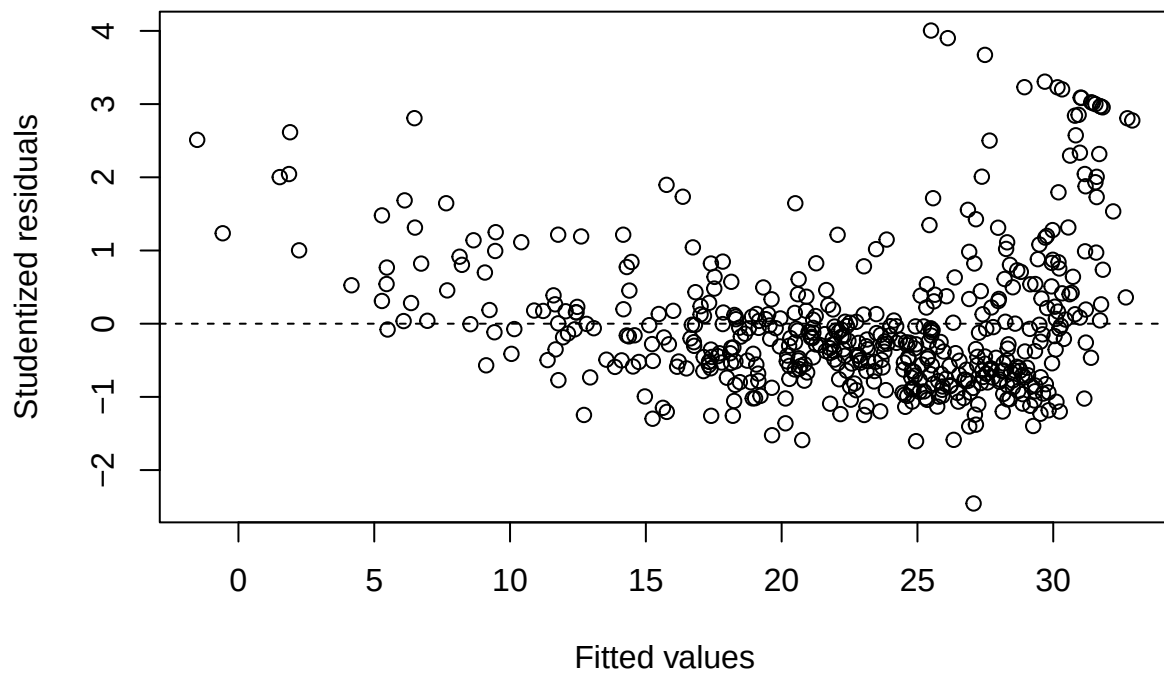


We can also compute residuals and studentized residuals manually, and inspect leverage statistics:

```
par(mfrow = c(1, 1))
plot(predict(lm.fit), residuals(lm.fit),
      xlab = "Fitted values", ylab = "Residuals")
abline(h = 0, lty = 2)
```

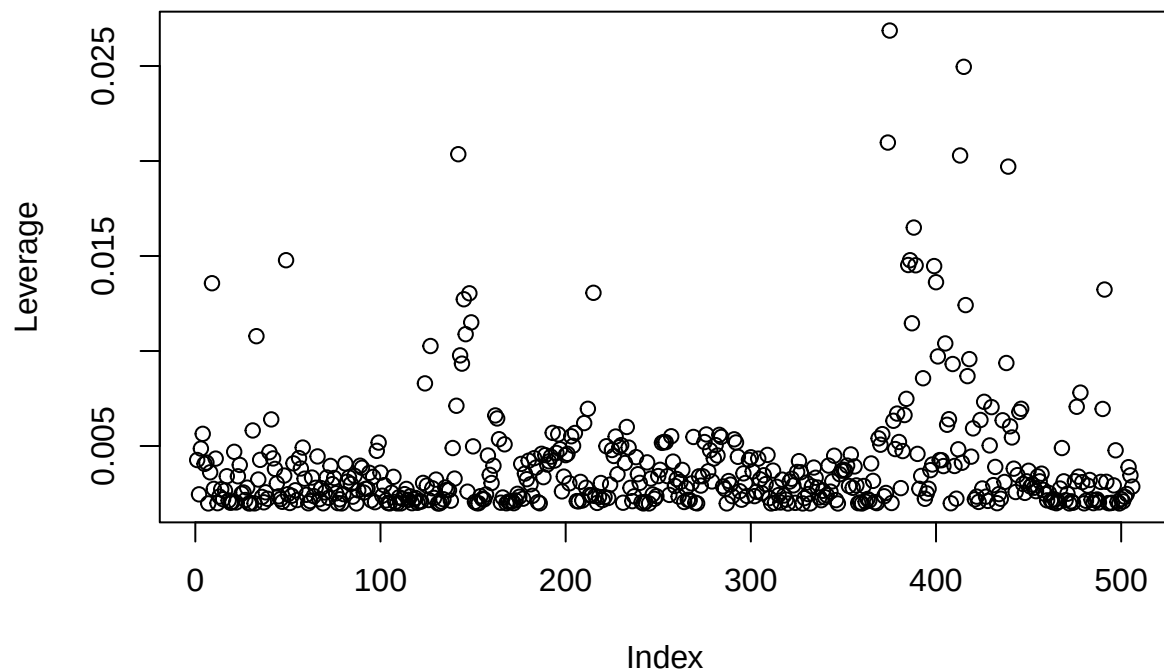


```
plot(predict(lm.fit), rstudent(lm.fit),
      xlab = "Fitted values", ylab = "Studentized residuals")
abline(h = 0, lty = 2)
```



```
# Leverage statistics -- which observations have unusually high leverage?
plot(hatvalues(lm.fit),
     ylab = "Leverage", main = "Leverage by observation")
```

Leverage by observation



```
which.max(hatvalues(lm.fit))
```

```
## 375
```

```
## 375
```

3 Multiple Linear Regression

A single predictor rarely tells the whole story — `medv` plausibly depends on more than just `lstat`. Multiple linear regression extends the same least-squares idea to p predictors:

$$\text{medv} = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \epsilon$$

Each $\hat{\beta}_j$ is now interpreted as the effect of a one-unit increase in X_j *holding all other predictors fixed* — a different reading than in simple regression, where $\hat{\beta}_1$ captured X_1 's effect with nothing else in the model to hold constant.

Now we extend the model to include more than one predictor.

```
lm.fit2 <- lm(medv ~ lstat + age, data = Boston)
summary(lm.fit2)
```

```
##
## Call:
## lm(formula = medv ~ lstat + age, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.981  -3.978  -1.283   1.968  23.158
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  33.22276    0.73085  45.458  < 2e-16 ***
## lstat       -1.03207    0.04819 -21.416  < 2e-16 ***
## age          0.03454    0.01223   2.826  0.00491 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.173 on 503 degrees of freedom
## Multiple R-squared:  0.5513, Adjusted R-squared:  0.5495
## F-statistic: 309 on 2 and 503 DF,  p-value: < 2.2e-16
```

3.1 Using all predictors

Rather than typing every variable name, we can regress `medv` on all other variables in the data frame using `.`:

```
lm.fit.all <- lm(medv ~ ., data = Boston)
summary(lm.fit.all)
```

```
##
## Call:
## lm(formula = medv ~ ., data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.1304  -2.7673  -0.5814   1.9414  26.2526
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  41.617270   4.936039   8.431 3.79e-16 ***
## crim        -0.121389   0.033000  -3.678 0.000261 ***
## zn          0.046963   0.013879   3.384 0.000772 ***
## indus       0.013468   0.062145   0.217 0.828520
## chas        2.839993   0.870007   3.264 0.001173 **
## nox       -18.758022   3.851355  -4.870 1.50e-06 ***
## rm          3.658119   0.420246   8.705 < 2e-16 ***
## age         0.003611   0.013329   0.271 0.786595
## dis        -1.490754   0.201623  -7.394 6.17e-13 ***
## rad         0.289405   0.066908   4.325 1.84e-05 ***
## tax        -0.012682   0.003801  -3.337 0.000912 ***
## ptratio    -0.937533   0.132206  -7.091 4.63e-12 ***
## lstat      -0.552019   0.050659 -10.897 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.798 on 493 degrees of freedom
## Multiple R-squared:  0.7343, Adjusted R-squared:  0.7278
## F-statistic: 113.5 on 12 and 493 DF,  p-value: < 2.2e-16
```

We can access individual pieces of the summary object, such as R^2 and the residual standard error:

```
summary(lm.fit.all)$r.squared
```

```
## [1] 0.734307
```

```
summary(lm.fit.all)$sigma
```

```
## [1] 4.798034
```

3.2 Checking for multicollinearity

Variance inflation factors (VIFs) help flag predictors that are highly correlated with other predictors, which can inflate coefficient standard errors. For predictor X_j , the VIF is defined using $R^2_{X_j}$ — the R^2 from regressing X_j on *all the other predictors*:

$$\text{VIF}(\hat{\beta}_j) = \frac{1}{1 - R^2_{X_j}}$$

If X_j is well explained by the other predictors ($R^2_{X_j}$ close to 1), the denominator shrinks toward 0

and the VIF blows up — a sign that X_j 's own coefficient estimate is unstable because the data can't cleanly separate its effect from the other predictors it's correlated with. A VIF of 1 means no collinearity at all; VIFs above roughly 5–10 are typically treated as cause for concern.

```
vif(lm.fit.all)

##      crim      zn      indus      chas      nox      rm      age      dis
## 1.767486 2.298459 3.987181 1.071168 4.369093 1.912532 3.088232 3.954037
##      rad      tax ptratio      lstat
## 7.445301 9.002158 1.797060 2.870777
```

3.3 Fitting with one predictor excluded

Suppose we want to fit the model using all predictors except one (say, `age`, which had a very high p-value above). We can do this two ways:

```
lm.fit.no.age <- lm(medv ~ . - age, data = Boston)
summary(lm.fit.no.age)

##
## Call:
## lm(formula = medv ~ . - age, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.1851  -2.7330  -0.6116   1.8555  26.3838
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  41.525128   4.919684   8.441 3.52e-16 ***
## crim        -0.121426   0.032969  -3.683 0.000256 ***
## zn           0.046512   0.013766   3.379 0.000785 ***
## indus        0.013451   0.062086   0.217 0.828577
## chas         2.852773   0.867912   3.287 0.001085 **
## nox        -18.485070   3.713714  -4.978 8.91e-07 ***
## rm           3.681070   0.411230   8.951 < 2e-16 ***
## dis         -1.506777   0.192570  -7.825 3.12e-14 ***
## rad          0.287940   0.066627   4.322 1.87e-05 ***
## tax         -0.012653   0.003796  -3.333 0.000923 ***
## ptratio     -0.934649   0.131653  -7.099 4.39e-12 ***
## lstat       -0.547409   0.047669 -11.483 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.794 on 494 degrees of freedom
## Multiple R-squared:  0.7343, Adjusted R-squared:  0.7284
## F-statistic: 124.1 on 11 and 494 DF, p-value: < 2.2e-16

# Equivalently, using update() on an existing fit:
lm.fit.updated <- update(lm.fit.all, ~ . - age)
```

```
summary(lm.fit.updated)
```

```
##
## Call:
## lm(formula = medv ~ crim + zn + indus + chas + nox + rm + dis +
##      rad + tax + ptratio + lstat, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.1851  -2.7330  -0.6116   1.8555  26.3838
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  41.525128   4.919684   8.441 3.52e-16 ***
## crim        -0.121426   0.032969  -3.683 0.000256 ***
## zn           0.046512   0.013766   3.379 0.000785 ***
## indus        0.013451   0.062086   0.217 0.828577
## chas         2.852773   0.867912   3.287 0.001085 **
## nox        -18.485070   3.713714  -4.978 8.91e-07 ***
## rm           3.681070   0.411230   8.951 < 2e-16 ***
## dis        -1.506777   0.192570  -7.825 3.12e-14 ***
## rad          0.287940   0.066627   4.322 1.87e-05 ***
## tax        -0.012653   0.003796  -3.333 0.000923 ***
## ptratio     -0.934649   0.131653  -7.099 4.39e-12 ***
## lstat       -0.547409   0.047669 -11.483 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.794 on 494 degrees of freedom
## Multiple R-squared:  0.7343, Adjusted R-squared:  0.7284
## F-statistic: 124.1 on 11 and 494 DF,  p-value: < 2.2e-16
```

4 Interaction Terms

Additive multiple regression assumes each predictor's effect is independent of the others — the effect of `lstat` on `medv` is the same whichever value `age` happens to take. An interaction term relaxes that assumption:

$$\text{medv} = \beta_0 + \beta_1 \cdot \text{lstat} + \beta_2 \cdot \text{age} + \beta_3 \cdot (\text{lstat} \times \text{age}) + \epsilon$$

Rearranging the `lstat` terms makes the point concrete — `lstat`'s effective slope now depends on the value of `age`:

$$\text{medv} = \beta_0 + (\beta_1 + \beta_3 \cdot \text{age}) \cdot \text{lstat} + \beta_2 \cdot \text{age} + \epsilon$$

`lstat:age` includes only the interaction term (β_3), while `lstat*age` is shorthand for `lstat + age`

+ `lstat:age` — i.e., it includes both main effects and the interaction, matching the full formula above.

```
summary(lm(medv ~ lstat * age, data = Boston))

##
## Call:
## lm(formula = medv ~ lstat * age, data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.806  -4.045  -1.333   2.085  27.552
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) 36.0885359  1.4698355  24.553  < 2e-16 ***
## lstat       -1.3921168  0.1674555  -8.313 8.78e-16 ***
## age         -0.0007209  0.0198792  -0.036  0.9711
## lstat:age    0.0041560  0.0018518   2.244  0.0252 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.149 on 502 degrees of freedom
## Multiple R-squared:  0.5557, Adjusted R-squared:  0.5531
## F-statistic: 209.3 on 3 and 502 DF,  p-value: < 2.2e-16
```

5 Non-linear Transformations of Predictors

The `lm()` function can accommodate non-linear transformations of predictors directly in the formula, using `I()` to protect arithmetic operators like `^` from being interpreted as formula syntax.

$$\text{medv} = \beta_0 + \beta_1 \cdot \text{lstat} + \beta_2 \cdot \text{lstat}^2 + \epsilon$$

This is still a *linear* model in the statistical sense — linear in the parameters $\beta_0, \beta_1, \beta_2$ — even though the fitted curve itself is not a straight line. That’s exactly why `lm()` can fit it directly.

```
lm.fit.sq <- lm(medv ~ lstat + I(lstat^2), data = Boston)
summary(lm.fit.sq)
```

```
##
## Call:
## lm(formula = medv ~ lstat + I(lstat^2), data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.2834  -3.8313  -0.5295   2.3095  25.4148
##
## Coefficients:
```

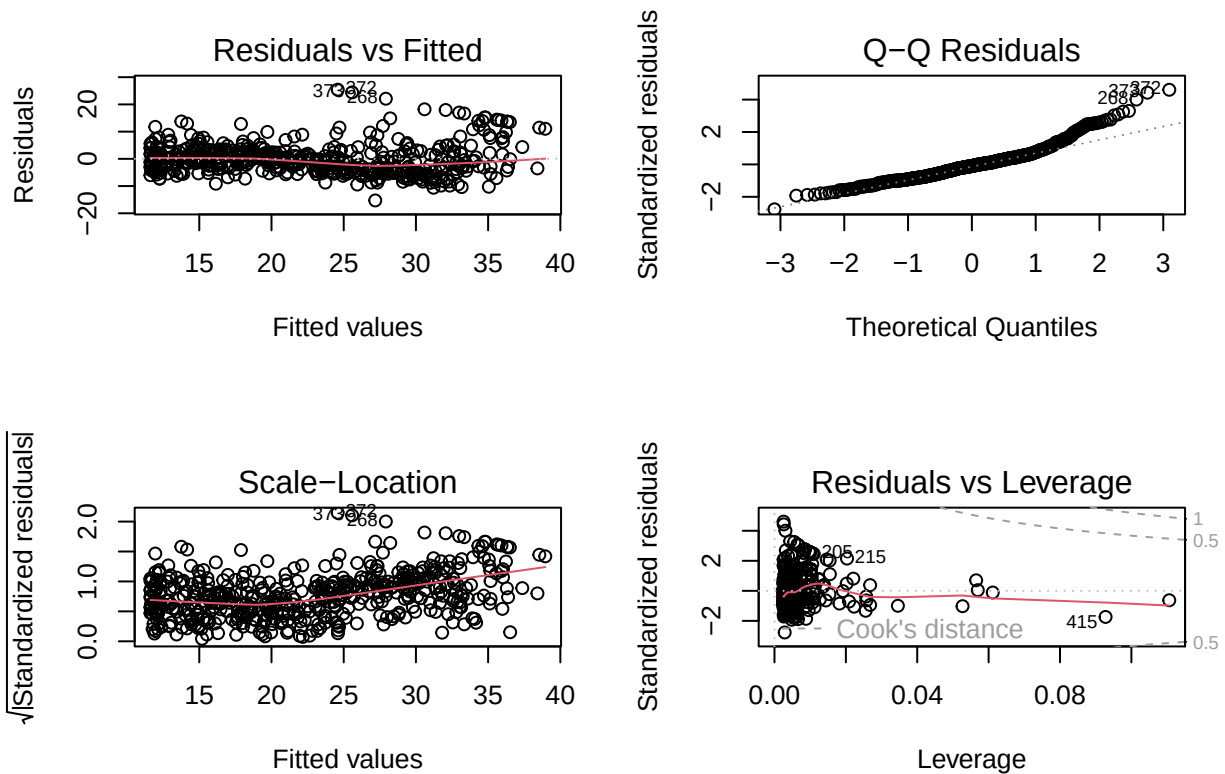
```
##           Estimate Std. Error t value Pr(>|t|)
## (Intercept) 42.862007   0.872084   49.15  <2e-16 ***
## lstat       -2.332821   0.123803  -18.84  <2e-16 ***
## I(lstat^2)   0.043547   0.003745   11.63  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.524 on 503 degrees of freedom
## Multiple R-squared:  0.6407, Adjusted R-squared:  0.6393
## F-statistic: 448.5 on 2 and 503 DF,  p-value: < 2.2e-16
```

We can use `anova()` to formally compare the linear-only model to the model that also includes the quadratic term:

```
lm.fit.linear <- lm(medv ~ lstat, data = Boston)
anova(lm.fit.linear, lm.fit.sq)
```

```
## Analysis of Variance Table
##
## Model 1: medv ~ lstat
## Model 2: medv ~ lstat + I(lstat^2)
##   Res.Df    RSS Df Sum of Sq    F    Pr(>F)
## 1     504 19472
## 2     503 15347   1    4125.1 135.2 < 2.2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

par(mfrow = c(2, 2))
plot(lm.fit.sq)
```



For a higher-order polynomial fit, the `poly()` function is more convenient than writing out each $I(lstat^k)$ term by hand:

```
lm.fit.poly5 <- lm(medv ~ poly(lstat, 5), data = Boston)
summary(lm.fit.poly5)
```

```
##
## Call:
## lm(formula = medv ~ poly(lstat, 5), data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -13.5433  -3.1039  -0.7052   2.0844  27.1153
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    22.5328     0.2318  97.197 < 2e-16 ***
## poly(lstat, 5)1 -152.4595     5.2148 -29.236 < 2e-16 ***
## poly(lstat, 5)2   64.2272     5.2148  12.316 < 2e-16 ***
## poly(lstat, 5)3  -27.0511     5.2148  -5.187 3.10e-07 ***
## poly(lstat, 5)4   25.4517     5.2148   4.881 1.42e-06 ***
## poly(lstat, 5)5  -19.2524     5.2148  -3.692 0.000247 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.215 on 500 degrees of freedom
```

```
## Multiple R-squared:  0.6817, Adjusted R-squared:  0.6785
## F-statistic: 214.2 on 5 and 500 DF,  p-value: < 2.2e-16
```

We can also try a log transformation of a predictor:

```
summary(lm(medv ~ log(rm), data = Boston))
```

```
##
## Call:
## lm(formula = medv ~ log(rm), data = Boston)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -19.487  -2.875  -0.104   2.837  39.816
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -76.488      5.028  -15.21  <2e-16 ***
## log(rm)       54.055      2.739   19.73  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 6.915 on 504 degrees of freedom
## Multiple R-squared:  0.4358, Adjusted R-squared:  0.4347
## F-statistic: 389.3 on 1 and 504 DF,  p-value: < 2.2e-16
```

6 Qualitative Predictors

We switch to the `Carseats` data set, which includes a mix of quantitative and qualitative (categorical) predictors, to demonstrate how `lm()` handles factor variables.

```
Carseats <- ISLR2::Carseats
str(Carseats)
```

```
## 'data.frame':    400 obs. of  11 variables:
## $ Sales      : num  9.5 11.22 10.06 7.4 4.15 ...
## $ CompPrice  : num  138 111 113 117 141 124 115 136 132 132 ...
## $ Income     : num  73 48 35 100 64 113 105 81 110 113 ...
## $ Advertising: num  11 16 10 4 3 13 0 15 0 0 ...
## $ Population : num  276 260 269 466 340 501 45 425 108 131 ...
## $ Price      : num  120 83 80 97 128 72 108 120 124 124 ...
## $ ShelfLoc   : Factor w/ 3 levels "Bad","Good","Medium": 1 2 3 3 1 1 3 2 3 3 ...
## $ Age        : num  42 65 59 55 38 78 71 67 76 76 ...
## $ Education  : num  17 10 12 14 13 16 15 10 10 17 ...
## $ Urban      : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 1 2 2 1 1 ...
## $ US         : Factor w/ 2 levels "No","Yes": 2 2 2 2 1 2 1 2 1 2 ...
```

6.1 Fitting a model with interactions and a factor variable

```
lm.fit.carseats <- lm(Sales ~ . + Income:Advertising + Price:Age,
                      data = Carseats)
summary(lm.fit.carseats)

##
## Call:
## lm(formula = Sales ~ . + Income:Advertising + Price:Age, data = Carseats)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.9208 -0.7503  0.0177  0.6754  3.3413
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    6.5755654   1.0087470    6.519 2.22e-10 ***
## CompPrice      0.0929371   0.0041183   22.567 < 2e-16 ***
## Income         0.0108940   0.0026044    4.183 3.57e-05 ***
## Advertising    0.0702462   0.0226091    3.107 0.002030 **
## Population     0.0001592   0.0003679    0.433 0.665330
## Price        -0.1008064   0.0074399  -13.549 < 2e-16 ***
## ShelveLocGood   4.8486762   0.1528378   31.724 < 2e-16 ***
## ShelveLocMedium 1.9532620   0.1257682   15.531 < 2e-16 ***
## Age           -0.0579466   0.0159506   -3.633 0.000318 ***
## Education     -0.0208525   0.0196131   -1.063 0.288361
## UrbanYes       0.1401597   0.1124019    1.247 0.213171
## USYes         -0.1575571   0.1489234   -1.058 0.290729
## Income:Advertising 0.0007510  0.0002784    2.698 0.007290 **
## Price:Age      0.0001068   0.0001333    0.801 0.423812
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.011 on 386 degrees of freedom
## Multiple R-squared:  0.8761, Adjusted R-squared:  0.8719
## F-statistic: 210 on 13 and 386 DF, p-value: < 2.2e-16
```

R automatically creates dummy variables for qualitative predictors such as `ShelveLoc` (a three-level factor: `Bad`, `Medium`, `Good`). With $k = 3$ levels, R creates $k - 1 = 2$ dummy variables and folds their coefficients into the same linear model:

$$\text{Sales} = \beta_0 + \beta_1 \cdot X_{\text{Good}} + \beta_2 \cdot X_{\text{Medium}} + \cdots + \epsilon, \quad X_{\text{Good}} = \begin{cases} 1 & \text{if ShelveLoc} = \text{Good} \\ 0 & \text{otherwise} \end{cases}$$

(and similarly for X_{Medium}). The omitted level, `Bad`, becomes the baseline: when both dummies are 0, the model's intercept β_0 represents a `Bad` shelf location, so β_1 and β_2 are read as the shift in expected `Sales` *relative to Bad*, not as standalone effects. We can check how these dummy variables were coded using `contrasts()`:

```
contrasts(Carseats$ShelveLoc)
```

```
##           Good Medium
## Bad           0      0
## Good          1      0
## Medium        0      1
```

The coefficients on `ShelveLocGood` and `ShelveLocMedium` represent the expected change in sales relative to the baseline level (Bad shelf location), holding all other predictors fixed.

7 Writing a Small Function

Finally, a short example of writing our own function – here, one that loads several packages at once, which can be handy at the start of a script or report like this one.

```
LoadLibraries <- function() {
  library(ISLR2)
  library(MASS)
  library(car)
  print("The libraries have been loaded.")
}

LoadLibraries()
```

```
## [1] "The libraries have been loaded."
```

8 Summary

In this lab we:

1. Fit and interpreted simple and multiple linear regression models.
2. Computed confidence and prediction intervals for predicted responses.
3. Produced and interpreted standard regression diagnostic plots.
4. Checked for multicollinearity using variance inflation factors.
5. Added interaction terms and non-linear (polynomial, log) transformations.
6. Incorporated qualitative predictors and inspected their dummy-variable coding.

These tools form the foundation for the more flexible modeling approaches covered in later chapters.